

Timsort

Group 5

Contents

Introduction	2
Methodology	3
Sorting algorithms to compare against	3
Notable metrics	3
Parameters	4
Data generation	4
Computer Specifications	4
Data Collection Methods	7
Time complexity analysis	8
Performance results	9
Runtime	9
Comparisons	11
Swaps	12
Copies (excluding swaps)	12
Copies (including swaps)	13
Conclusion	15
• [] Algorithm explanation with examples	
• [] Time complexity analysis	
• [] Java implementation	
☒ Testing methodology	
• [] Performance results for 5, 1,000, and 1,000,000 elements	
☒ Comparison with Bubble Sort, Selection Sort, and Insertion Sort	
• [] Appropriate and inappropriate use cases	
• [] Real-world applications	
• [] Conclusion and findings	

Introduction

Timsort is a hybrid, stable sorting algorithm, derived from merge sort and insertion sort, designed to perform well on many kinds of real-world data. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds subsequences of the data that are already ordered (runs) and uses them to sort the remainder more efficiently.

Methodology

We will mostly be empirically determining Tim sort's performance through sample trials and data analysis. We will then see whether the data matches with the theoretical time complexity of the algorithm.

We will then compare it against other sorting algorithms to see why it is one of the most implemented sorting algorithm.

Sorting algorithms to compare against

We will compare it against the following sorting algorithms:

- Bubble sort
- Insertion sort
- Selection sort
- Merge sort
- Timsort
- Java's sort

The reason why we included Merge sort is because Timsort builds off of Merge sort. Moreover, our "control" will be Java's sorting algorithm.

Notable metrics

For every trial run, we will keep track of the following metrics:

- runtime
- number of comparisons
- number of swaps
- number of copies (excluding swaps)
- number of copies (including swaps)

Swap consists of 3 copies, and since some algorithms don't really use conventional swapping, we believe it's a better metric to compare in place of the amount of swaps. In the code, the `copies` variable is the amount of copies excluding the amount of swaps.

Parameters

Tim sort has a adjustable parameter, which is the minimum run length. For our particular use case, we will use a minimum run size of 32.

Data generation

The most obvious data set we should use is integers, however, that begs the question, what should be the range of the randomly generated integers?

Suppose that we generate random integers in the range between $[1, k]$ and we generate a list of size n . If $k \gg n$, then we would rarely get duplicates, meanwhile, if $k \ll n$, then we would get many duplicates.

Our parameter will then be the ratio $r = \frac{n}{k}$ where a high value of r would mean more duplicates (I think the duplicate can be thought of as the expected number of duplicates, although, I'm not too certain on the probabilistic analysis on that).

In our case, we selected 2 duplication ratios:

- $r = 10$ for data with high duplication
- $r = 0.1$ for data with low duplication

For each duplication ratios, we will use datasets of the following sizes:

- $n = 5$ for small datasets (for data with high duplication, $k = 1$, meaning that the data will just be an array containing just 1)
- $n = 1000$ for medium sized datasets
- $n = 1000000$ for large datasets

Computer Specifications

To make sure that the run time results aren't too much affected by other external factors, we ran the same algorithms on the same computer.

The most important variable for run time in this case is probably the CPU, and the speed and size of the RAM.

In our case, we used a CPU with the following `lscpu` output:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 6
On-line CPU(s) list:    0-5
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i5-9500 CPU @ 3.00GHz
CPU family:             6
```

Model:	158
Thread(s) per core:	1
Core(s) per socket:	6
Socket(s):	1
Stepping:	10
Microcode version:	0xfa
CPU(s) scaling MHz:	20%
CPU max MHz:	4400.0000
CPU min MHz:	800.0000
BogoMIPS:	6000.00
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid mpx rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp vnmi md_clear flush_l1d arch_capabilities
Virtualization features:	
Virtualization:	VT-x
Caches (sum of all):	
L1d:	192 KiB (6 instances)
L1i:	192 KiB (6 instances)
L2:	1.5 MiB (6 instances)
L3:	9 MiB (1 instance)
NUMA:	
NUMA node(s):	1
NUMA node0 CPU(s):	0-5
Vulnerabilities:	
Gather data sampling:	Mitigation; Microcode
Ghostwrite:	Not affected
Indirect target selection:	Not affected
Itlb multihit:	KVM: Mitigation: Split huge pages
L1tf:	Mitigation; PTE Inversion; VMX conditional cache flushes, SMT disabled
Mds:	Mitigation; Clear CPU buffers; SMT disabled
Meltdown:	Mitigation; PTI
Mmio stale data:	Mitigation; Clear CPU buffers; SMT disabled
Old microcode:	Not affected
Reg file data sampling:	Not affected

Retbleed:	Mitigation; IBRS
Spec rstack overflow:	Not affected
Spec store bypass:	Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:	Mitigation; usercopy/swaps barriers and __user pointer sanitization
Spectre v2:	Mitigation; IBRS; IBPB conditional; STIBP disabled; RSB filling; PBRSE-IBRS Not affected; BHI Not affected
Srbds:	Mitigation; Microcode
Tsa:	Not affected
Tsx async abort:	Mitigation; TSX disabled
Vmscape:	Mitigation; IBPB before exit to userspace

As for the RAM, the output of `sudo dmidecode --type 17` is as follows:

Getting SMBIOS data from sysfs.
SMBIOS 3.1.1 present.

Handle 0x000A, DMI type 17, 40 bytes

Memory Device

```

    Array Handle: 0x0009
    Error Information Handle: Not Provided
    Total Width: 64 bits
    Data Width: 64 bits
    Size: 8 GiB
    Form Factor: DIMM
    Set: None
    Locator: DIMM1
    Bank Locator: Not Specified
    Type: DDR4
    Type Detail: Synchronous
    Speed: 2666 MT/s
    Manufacturer: 80CE000080CE
    Serial Number: 343C8B13
    Asset Tag: 02193900
    Part Number: M378A1K43DB2-CTD
    Rank: 1
    Configured Memory Speed: 2666 MT/s
    Minimum Voltage: Unknown
    Maximum Voltage: Unknown
    Configured Voltage: 1.2 V

```

Handle 0x000B, DMI type 17, 40 bytes

Memory Device

```

    Array Handle: 0x0009
    Error Information Handle: Not Provided
    Total Width: Unknown
    Data Width: Unknown

```

Size: No Module Installed
Form Factor: Unknown
Set: None
Locator: DIMM2
Bank Locator: Not Specified
Type: Unknown
Type Detail: None

The OS might also be a factor. We used a Linux kernel which has a `uname -a` output as follows:

7.2.3-arch1-3

The test would be decently fair when tested one after the other because the CPU and memory utilization for other processes would be similar.

Data Collection Methods

Because of the randomness of the dataset, and the randomness of OS scheduling, it is necessary to run the code multiple time and record the average of all notable metrics alongside with its standard deviation.

We will run perform each run 100 times to ensure a good sample size.

Time complexity analysis

Performance results

Note that the $\pm$ uncertainty range is in terms of the sample standard deviation σ . If we assume that it's normally distributed, we can estimate that 99.7% of the sample falls in between $\mu \pm 3 \times \sigma$, where μ is the sample mean. If we remain conservative and assume that the data is of an arbitrary distribution, Chebyshev's theorem gives us that 99% of the samples must fall in between the range of $\mu \pm 10 \times \sigma$.

Also note that we will refer for the array of sized 5 as small sized, the array of sized 1000 as medium sized, and the array of sized 1000000 as large sized.

Runtime

Note that the numbers are in seconds.

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	$1.13 \times 10^{-5} \pm 7.90 \times 10^{-5}$	$2.31 \times 10^{-3} \pm 9.53 \times 10^{-4}$	N/A
insertionSort	$5.35 \times 10^{-6} \pm 7.69 \times 10^{-6}$	$8.43 \times 10^{-4} \pm 7.57 \times 10^{-4}$	N/A
javaSort	$4.09 \times 10^{-4} \pm 4.06 \times 10^{-3}$	$2.28 \times 10^{-4} \pm 8.95 \times 10^{-5}$	$2.68 \times 10^{-1} \pm 2.25 \times 10^{-2}$
mergeSort	$5.06 \times 10^{-6} \pm 8.06 \times 10^{-6}$	$6.32 \times 10^{-4} \pm 1.84 \times 10^{-4}$	$4.53 \times 10^{-1} \pm 1.41 \times 10^{-2}$
selectionSort	$3.90 \times 10^{-6} \pm 6.84 \times 10^{-6}$	$1.24 \times 10^{-3} \pm 8.31 \times 10^{-4}$	N/A
timSort	$1.11 \times 10^{-5} \pm 4.69 \times 10^{-5}$	$2.90 \times 10^{-4} \pm 9.38 \times 10^{-5}$	$4.55 \times 10^{-1} \pm 2.59 \times 10^{-2}$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	$3.29 \times 10^{-6} \pm$	$3.09 \times 10^{-3} \pm$	N/A
	7.40×10^{-6}	1.01×10^{-3}	
insertionSort	$3.24 \times 10^{-6} \pm$	$8.74 \times 10^{-4} \pm$	N/A
	6.32×10^{-6}	6.65×10^{-4}	
javaSort	$1.47 \times 10^{-6} \pm$	$9.27 \times 10^{-5} \pm$	$2.73 \times 10^{-1} \pm$
	4.62×10^{-6}	1.41×10^{-5}	2.41×10^{-2}
mergeSort	$4.87 \times 10^{-7} \pm$	$1.70 \times 10^{-4} \pm$	$4.50 \times 10^{-1} \pm$
	9.35×10^{-8}	1.24×10^{-5}	2.07×10^{-2}
selectionSort	$4.73 \times 10^{-6} \pm$	$1.31 \times 10^{-3} \pm$	N/A
	1.13×10^{-5}	8.29×10^{-4}	
timSort	$2.45 \times 10^{-6} \pm$	$1.69 \times 10^{-4} \pm$	$4.39 \times 10^{-1} \pm$
	4.37×10^{-6}	1.82×10^{-5}	1.03×10^{-2}

The reason why some of the values for the 1000000 column is missing is because, those are $O(n^2)$ sorting algorithms, as such, they wouldn't be able to finish in a reasonable amount of time.

This can easily be seen with some back-of-the-envelope calculation.

When we move from an input size of 1000 to 1000000, we're increasing the input size by a factor of 1000. As such, the amount of time it would take would be scaled by a factor of $1000^2 = 10^6$ or a million.

Since it took $2.31 \times 10^{-3}s$ for bubble sort to sort a sized 1000 array, for a sized 10^6 array, we would expect the algorithm to take take:

$$2.31 \times 10^{-3} \times 10^6 = 2.31 \times 10^3 = 2310s$$

It would then take around 38.5 minutes to complete. If we were to run this 100 times, it would then take around 3850 minutes to complete, which is around 64.2 hours. As such, for $O(n^2)$ sorting algorithms, which in our case, include bubble sort, selection sort, and insertion sort; they won't have an entry in the array of sized 1000000 column of any of the other metrics either.

For data with low duplication, Timsort and merge sort performs very similarly for large input (10^6 length array), although, it's about 2 times slower than Java's implementation of `Arrays.sort`. This is somewhat expected since for data with low duplication, most of the runs will be of size `minRun`, which makes it behave similarly to merge sort.

Where Timsort runs somewhat faster than merge sort in the case of data with low duplication with sized 1000, although, it might not be a statistically significant difference given the uncertainty.

As for the data with high duplication, the results doesn't seem significantly different, perhaps because the duplication ratio is too low.

All in all, for our particular dataset, Timsort performs very similarly to merge sort, and since Timsort is $O(n \log_2 n)$, it can sort an array of sized 1000000 in a reasonable amount of time compared to bubble sort, selection sort, and insertion sort.

Comparisons

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	10 ± 0	$4.995 \times 10^5 \pm 0$	N/A
insertionSort	13.47 ± 3.00	$1.906 \times 10^4 \pm 44.92$	N/A
javaSort	N/A	N/A	N/A
mergeSort	7.18 ± 0.85	$8.709 \times 10^3 \pm 16.35$	$1.867 \times 10^7 \pm 555.35$
selectionSort	14 ± 0	$5.005 \times 10^5 \pm 0$	N/A
timSort	16.35 ± 2.96	$1.342 \times 10^4 \pm 67.41$	$2.343 \times 10^7 \pm 1.627 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	10 ± 0	$4.995 \times 10^5 \pm 0$	N/A
insertionSort	4 ± 0	$1.894 \times 10^4 \pm 71.26$	N/A
javaSort	N/A	N/A	N/A
mergeSort	7 ± 0	$8.701 \times 10^3 \pm 16.12$	$1.867 \times 10^7 \pm 611.02$
selectionSort	14 ± 0	$5.005 \times 10^5 \pm 0$	N/A
timSort	4 ± 0	$1.337 \times 10^4 \pm 75.44$	$2.343 \times 10^7 \pm 2.075 \times 10^3$

Note that for Java's sort, we couldn't count the amount of swaps, nor the amount of comparisons because they don't use our particular implementation of swap and comparison which has innate metrics logging.

The amount of comparisons has a standard deviation that's quite tight, which is good for determining the difference between different algorithms.

For the medium sized array, Timsort has fewer comparisons than insertion sort, however, merge sort has a significantly lower amount of comparisons. This is probably due to the fact that Timsort requires comparisons for merging similar to merge sort, but also comparisons to determine the amount of runs, whether it's decreasing, and because we use insertion sort for smaller sized array.

Swaps

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	4.92 ± 1.98	$2.492 \times 10^5 \pm 5.133 \times 10^3$	N/A
insertionSort	0 ± 0	0 ± 0	N/A
javaSort	N/A	N/A	N/A
mergeSort	0 ± 0	0 ± 0	0 ± 0
selectionSort	2.97 ± 0.66	992.43 ± 2.36	N/A
timSort	1.14 ± 1.00	246.72 ± 42.51	$2.500 \times 10^5 \pm 1.366 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	$2.467 \times 10^5 \pm 4.822 \times 10^3$	N/A
insertionSort	0 ± 0	0 ± 0	N/A
javaSort	N/A	N/A	N/A
mergeSort	0 ± 0	0 ± 0	0 ± 0
selectionSort	3 ± 0	993.42 ± 2.21	N/A
timSort	0 ± 0	247.52 ± 44.52	$2.499 \times 10^5 \pm 1.244 \times 10^3$

Bubble sort and selection sort requires swaps, however, insertion sort and merge sort doesn't do a conventional swap, as such, this isn't quite a fair metric of comparison.

Copies (excluding swaps)

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	0 ± 0	N/A
insertionSort	11.04 ± 3.38	$2.517 \times 10^5 \pm 4.861 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	0 ± 0	0 ± 0	N/A

Algorithm	5	1000	1000000
timSort	8.03 ± 2.91	$1.940 \times 10^4 \pm 173.41$	$3.941 \times 10^7 \pm 5.546 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	0 ± 0	N/A
insertionSort	0 ± 0	$2.500 \times 10^5 \pm 4.928 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	0 ± 0	0 ± 0	N/A
timSort	0 ± 0	$1.929 \times 10^4 \pm 162.64$	$3.941 \times 10^7 \pm 5.402 \times 10^3$

The copies excluding swaps gets counted when we do things such as right shift operators and merging. Nonetheless, it still isn't a fair metric of comparison because, bubble sort and selection sort only uses swaps, which doesn't get counted into the copies excluding swaps.

Copies (including swaps)

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	14.76 ± 5.95	$7.477 \times 10^5 \pm 1.540 \times 10^4$	N/A
insertionSort	11.04 ± 3.38	$2.517 \times 10^5 \pm 4.861 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	8.91 ± 1.98	$2.977 \times 10^3 \pm 7.07$	N/A
timSort	11.45 ± 4.47	$2.014 \times 10^4 \pm 224.68$	$4.016 \times 10^7 \pm 6.637 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	$7.402 \times 10^5 \pm 1.447 \times 10^4$	N/A
insertionSort	0 ± 0	$2.500 \times 10^5 \pm 4.928 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	9 ± 0	$2.980 \times 10^3 \pm 6.64$	N/A
timSort	0 ± 0	$2.004 \times 10^4 \pm 205.08$	$4.016 \times 10^7 \pm 6.658 \times 10^3$

The copies which include swaps includes swaps into the total amount of copies, where one swap amounts to 3 copies operations.

For arrays with a high low duplication factor, bubble sort, and Timsort doesn't do any copy and exits for small arrays. The array consists of just 1, meaning that it is already sorted. Merge sort on the other hand is naive, and still splits the array, even though it's already sorted. Similarly for selection sort, it will still methodically search for the minimum and try to swap it, because it's naive.

For bigger arrays, for both arrays with low and high duplication factor, the amount of copies between merge sort and Timsort isn't significant.

Conclusion